

N-gram Language Models

Natural Language Processing

Daniel Ortiz Martínez

Table of Contents

1. Introduction
2. n -gram Language Models
3. Zipf's Law
4. Smoothing Techniques
5. Text Generation as Sequential Sampling
6. Practical Aspects when Working with n -gram Language Models
7. Evaluating Language Models
8. The Gutenberg and Brown Corpora
9. Summary

Introduction

What is a Language Model

- A language model is a probabilistic model that assigns probabilities to text or strings
- Examples of problems that benefit from language models:
 - Spelling correction
 - Machine translation
 - Speech recognition
 - Handwriting recognition
 - Word suggestions in text editors

What is a Language Model

- There are different ways to implement a language model
- One successful approach were the so-called n -gram language models
- N -gram language models follow a statistical approach
- Although n -gram language models are no longer the state of the art in language modeling, they are still in use and are very useful to illustrate the problem of language modeling and how a language model works

n-gram Language Models

From Probabilities to n -grams

- A language model must assign a probability to any word sequence $x = (w_1, w_2, \dots, w_N)$

- Using the chain rule of probability:

$$\begin{aligned} p(x) &= p(w_1) p(w_2|w_1) p(w_3|w_1, w_2) \cdots p(w_N|w_1, \dots, w_{N-1}) \\ &= \prod_{i=1}^N p(w_i | w_1, \dots, w_{i-1}) \end{aligned}$$

- The conditioning history $w_1, \dots, w_{i-1}$ grows with i (estimating these probabilities from a corpus becomes infeasible for long sequences)
- We need a way to limit the context

The Markov Assumption

- The k -th order Markov assumption truncates history to last k words:

$$p(w_i \mid w_1, \dots, w_{i-1}) \approx p(w_i \mid w_{i-k}, \dots, w_{i-1})$$

- An **n -gram model** is a $(n-1)$ -th order Markov model:

Model	Order	Approximation
Unigram	$k = 0$	$p(w_i)$
Bigram	$k = 1$	$p(w_i \mid w_{i-1})$
Trigram	$k = 2$	$p(w_i \mid w_{i-2}, w_{i-1})$
n -gram	$k = n-1$	$p(w_i \mid w_{i-n+1}, \dots, w_{i-1})$

- The sequence probability becomes:

$$p(x) \approx \prod_{i=1}^N p(w_i \mid w_{i-n+1}, \dots, w_{i-1})$$

Estimating n -gram Probabilities

- n -gram probabilities are estimated from a corpus by relative frequency counting:

$$p(w_i \mid w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_{i-1}, w_i)}{C(w_{i-n+1}, \dots, w_{i-1})}$$

- For example, for a bigram model:

$$p(\text{house} \mid \text{the}) = \frac{C(\text{the house})}{C(\text{the})}$$

- Larger n captures more context but requires exponentially more data to estimate reliably (a direct consequence of Zipf's law)

n -gram Language Models: Example

Let us consider a toy corpus from

https://en.wikipedia.org/wiki/This_Is_the_House_That_Jack_Built:

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

n-gram Language Models: Example

We want to compute a bi-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_i | w_{i-1}) = p(w_{i-1}, w_i) / p(w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

How could we estimate $p(w_i, w_j)$?

Divide the count of (w_i, w_j) in the corpus
by the count of w_j

n-gram Language Models: Example

We want to compute a bi-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_i | w_{i-1}) = p(w_{i-1}, w_i) / p(w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

How could we estimate $p(w_i, w_j)$?

Divide the count of (w_i, w_j) in the corpus
by the count of w_j

$$p(\text{house}|\text{the}) = \frac{\text{the house}}{\text{the}} = ?$$

n-gram Language Models: Example

We want to compute a bi-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_i | w_{i-1}) = p(w_{i-1}, w_i) / p(w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

How could we estimate $p(w_i, w_j)$?

Divide the count of (w_i, w_j) in the corpus
by the count of w_j

$$p(\text{house}|\text{the}) = \frac{\text{the house}}{\text{the}} = ?$$

n -gram Language Models: Example

We want to compute a bi-gram language model, which assumes

$$p(w_i|w_1, \dots, w_{i-1}) \approx p(w_i|w_{i-1}) = p(w_{i-1}, w_i)/p(w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

How could we estimate $p(w_i, w_j)$?

Divide the count of (w_i, w_j) in the corpus
by the count of w_j

$$p(\text{house}|\text{the}) = \frac{\text{the house}}{\text{the}} = \frac{4}{10}$$

n-gram Language Models: Example

We want to compute a four-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_{i-3}, w_{i-2}, w_{i-1}, w_i) / p(w_{i-3}, w_{i-2}, w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

$$p(\text{house} | \text{This, is, the}) = \frac{\text{This is the house}}{\text{This is the}} = ?$$

n -gram Language Models: Example

We want to compute a four-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_{i-3}, w_{i-2}, w_{i-1}, w_i) / p(w_{i-3}, w_{i-2}, w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

$$p(\text{house} | \text{This, is, the}) = \frac{\text{This is the house}}{\text{This is the}} = ?$$

n -gram Language Models: Example

We want to compute a four-gram language model, which assumes

$$p(w_i | w_1, \dots, w_{i-1}) \approx p(w_{i-3}, w_{i-2}, w_{i-1}, w_i) / p(w_{i-3}, w_{i-2}, w_{i-1})$$

This is the house that Jack built

This is the malt

That lay in the house that Jack built

This is the rat

That ate the malt

That lay in the house that Jack built

This is the cat

That killed the rat

That ate the malt

That lay in the house that Jack build

$$p(\text{house} | \text{This, is, the}) = \frac{\text{This is the house}}{\text{This is the}} = \frac{1}{4}$$

Language Model with $\langle s \rangle$ and $\langle /s \rangle$

- Symbols $\langle s \rangle$ and $\langle /s \rangle$ denote the start and stop of a sentence
- Using these special words will affect the model probabilities
- Consider the corpus:
 $\langle s \rangle$ This is the malt $\langle /s \rangle$
 $\langle s \rangle$ That lay in the house that Jack built $\langle /s \rangle$
- $p(\text{This is the house})$ for 2-gram language model without $\langle s \rangle$ and $\langle /s \rangle$:

$$p(\text{This is the house}) = p(\text{This})p(\text{is}|\text{This})p(\text{the}|\text{is})p(\text{house}|\text{the}) = \frac{1}{12} \cdot \frac{1}{1} \cdot \frac{1}{1} \cdot \frac{1}{2} = \frac{1}{24}$$

- $p(\text{This is the house})$ for 2-gram language model with $\langle s \rangle$ and $\langle /s \rangle$:

$$p(\langle s \rangle \text{ This is the house } \langle /s \rangle) = p(\text{This}|\langle s \rangle)p(\text{is}|\text{This})p(\text{the}|\text{is})p(\text{house}|\text{the})p(\langle /s \rangle|\text{house}) = ?$$

Language Model with $\langle s \rangle$ and $\langle /s \rangle$

- Symbols $\langle s \rangle$ and $\langle /s \rangle$ denote the start and stop of a sentence
- Using these special words will affect the model probabilities
- Consider the corpus:
 $\langle s \rangle$ This is the malt $\langle /s \rangle$
 $\langle s \rangle$ That lay in the house that Jack built $\langle /s \rangle$
- $p(\text{This is the house})$ for 2-gram language model without $\langle s \rangle$ and $\langle /s \rangle$:

$$p(\text{This is the house}) = p(\text{This})p(\text{is}|\text{This})p(\text{the}|\text{is})p(\text{house}|\text{the}) = \frac{1}{12} \cdot \frac{1}{1} \cdot \frac{1}{1} \cdot \frac{1}{2} = \frac{1}{24}$$

- $p(\text{This is the house})$ for 2-gram language model with $\langle s \rangle$ and $\langle /s \rangle$:

$$p(\langle s \rangle \text{ This is the house } \langle /s \rangle) = p(\text{This}|\langle s \rangle)p(\text{is}|\text{This})p(\text{the}|\text{is})p(\text{house}|\text{the})p(\langle /s \rangle|\text{house}) = \frac{1}{2} \cdot \frac{1}{1} \cdot \frac{1}{1} \cdot \frac{1}{2} \cdot 0 = 0$$

Zipf's Law

- The Zipf's law is an empirical law that may hold when a list of measured values is sorted in decreasing order
- The best known instance applies to the frequency table of words in a natural language text

$$\text{word frequency} \propto \frac{1}{\text{word rank}}$$

- Word rank refers to the position of a word within a frequency-ordered list of words in a text

- A Zipf-like distribution is obtained when representing word ranks against word frequencies
- In a Zipfian distribution, the most common word occurs approximately twice as often as the next common one, three times as often as the third most common, and so on

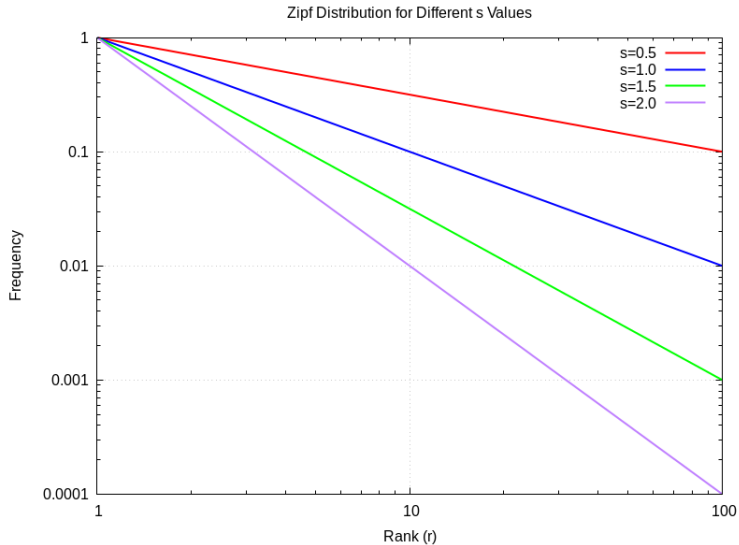
Zipf's Law: Distribution

$$P(r) = \frac{C}{r^s}$$

where:

- $P(r)$: probability (or frequency) of the element at rank r
- r : rank (with the most frequent item at $r = 1$)
- s : Zipf exponent, a positive real number, typically $s \approx 1$
- C : normalization constant ensuring that the sum of probabilities equals 1

Zipf's Law: Distribution



Zipf's Law and Unseen n -grams

- The Zipf's law determines that in a given text, there will be a few words that are very common and many words that are very uncommon
- This means that estimating accurate probabilities for rare words is challenging due to limited data

Smoothing Techniques

Dealing with Unseen n -grams: Smoothing

- Smoothing techniques are a key element in n -gram language models
- If we plot the probabilities of the events described by the model, smoothing techniques have the goal of making the distribution less sharp (or smoother)
- Smoothing tries to correct the problems derived of data scarcity and Zipf's law: a few common words will have too much probability and many words will have too few (or zero) probability

Dealing with Unseen n -grams: Laplace Smoothing

- We want to allow unseen grams to have non zero probability
- A common technique is called *Laplace smoothing* or *Add 1 smoothing*

$$p_{\text{add1}}(w_i|w_j) = \frac{c(w_j w_i) + 1}{c(w_j) + |V|} \quad p_{\text{add1}}(w_i) = \frac{c(w_i) + 1}{c() + |V|} = \frac{c(w_i) + 1}{\text{\#tokens} + |V|}$$

- Consider the corpus:
 < s > This is the malt < / s >
 < s > That lay in the house that Jack built < / s >
- With $V = \{\text{This, is, the, malt, That, lay, in, house, that, Jack, build, } \langle s \rangle, \langle /s \rangle\}$

$$p(\langle s \rangle \text{ This is the house } \langle /s \rangle) = p(\text{This}|\langle s \rangle)p(\text{is}|\text{This})p(\text{the}|\text{is})p(\text{house}|\text{the})p(\langle /s \rangle|\text{house}) =$$

Dealing with Unseen n -grams: Laplace Smoothing

- We want to allow unseen grams to have non zero probability
- A common technique is called *Laplace smoothing* or *Add 1 smoothing*

$$p_{\text{add1}}(w_i|w_j) = \frac{c(w_j w_i) + 1}{c(w_j) + |V|} \quad p_{\text{add1}}(w_i) = \frac{c(w_i) + 1}{c() + |V|} = \frac{c(w_i) + 1}{\text{\#tokens} + |V|}$$

- Consider the corpus:
<s> This is the malt </s>
<s> That lay in the house that Jack built </s>
- With $V = \{\text{This, is, the, malt, That, lay, in, house, that, Jack, build, <s>, </s>}\}$

$$p(< s > \text{ This is the house } < /s >) = p(\text{This} | < s >) p(\text{is} | \text{This}) p(\text{the} | \text{is}) p(\text{house} | \text{the}) p(< /s > | \text{house}) = \\ \frac{1+1}{2+13} \cdot \frac{1+1}{1+13} \cdot \frac{1+1}{1+13} \cdot \frac{1+1}{2+13} \cdot \frac{0+1}{1+13}$$

Dealing with Unseen n -grams: Lidstone Smoothing

- Laplace smoothing is a particular case of a more general technique:
Lidstone Smoothing

$$p_{\text{lidstone}}(w_i|w_j) = \frac{c(w_j w_i) + \epsilon}{c(w_j) + |V| \cdot \epsilon} \quad p_{\text{lidstone}}(w_i) = \frac{c(w_i) + \epsilon}{c() + |V| \cdot \epsilon} = \frac{c(w_i) + \epsilon}{\text{\#tokens} + |V| \cdot \epsilon}$$

- Consider the corpus:

<s> This is the malt </s>

<s> That lay in the house that Jack built </s>

- With $V = \{\text{This, is, the, malt, That, lay, in, house, that, Jack, build, <s>, </s>}\}$

$$p(< s > \text{ This is the house } < /s >) = p(\text{This} | < s >) p(\text{is} | \text{This}) p(\text{the} | \text{is}) p(\text{house} | \text{the}) p(< /s > | \text{house}) =$$

Dealing with Unseen n -grams: Lidstone Smoothing

- Laplace smoothing is a particular case of a more general technique:
Lidstone Smoothing

$$p_{\text{lidstone}}(w_i|w_j) = \frac{c(w_j w_i) + \epsilon}{c(w_j) + |V| \cdot \epsilon} \quad p_{\text{lidstone}}(w_i) = \frac{c(w_i) + \epsilon}{c() + |V| \cdot \epsilon} = \frac{c(w_i) + \epsilon}{\text{\#tokens} + |V| \cdot \epsilon}$$

- Consider the corpus:

<s> This is the malt </s>

<s> That lay in the house that Jack built </s>

- With $V = \{\text{This, is, the, malt, That, lay, in, house, that, Jack, build, <s>, </s>}\}$

$$p(< s > \text{ This is the house } < /s >) = p(\text{This} | < s >) p(\text{is} | \text{This}) p(\text{the} | \text{is}) p(\text{house} | \text{the}) p(< /s > | \text{house}) =$$
$$\frac{1 + 0.5}{2 + 13 \cdot 0.5} \cdot \frac{1 + 0.5}{1 + 13 \cdot 0.5} \cdot \frac{1 + 0.5}{1 + 13 \cdot 0.5} \cdot \frac{1 + 0.5}{2 + 13 \cdot 0.5} \cdot \frac{0 + 1}{1 + 13 \cdot 0.5} \quad \text{for } \epsilon = 0.5$$

- Lidstone smoothing is based on *discounting* probability mass from observed n -grams and redistributing it
 - the discounting is done by increasing the denominator of the relative frequency estimates
- An alternative smoothing technique is *backoff*
 - if a particular n -gram has not been seen, use $(n - 1)$ -grams

Backoff Models

Equation for Katz's backoff model*:

$$P_{\text{bo}}(w_i | w_{i-n+1} \dots w_{i-1}) = \begin{cases} d_{w_{i-n+1} \dots w_i} \frac{C(w_{i-n+1} \dots w_i)}{C(w_{i-n+1} \dots w_{i-1})} & \text{if } C(w_{i-n+1} \dots w_i) > k \\ \alpha_{w_{i-n+1} \dots w_{i-1}} P_{\text{bo}}(w_i | w_{i-n+2} \dots w_{i-1}) & \text{otherwise} \end{cases}$$

where:

- k is typically defined as zero
- d is the discounting amount using the so-called *Good-Turing* estimation
- α is another discounting amount called *backoff weight*

*Follow [this link](#) for a detailed explanation

Interpolation

- Interpolation is another way of combining different order n -gram models
- Instead of choosing probabilities for a particular n -gram order, interpolation takes the weighted average of multiple orders
- Example of interpolated trigram model[†]:

$$\begin{aligned} p_{\text{interpolation}}(w_i | w_{i-2}, w_{i-1}) &= \lambda_3 p_3^*(w_i | w_{i-2}, w_{i-1}) \\ &\quad + \lambda_2 p_2^*(w_i | w_{i-1}) \\ &\quad + \lambda_1 p_1^*(w_i) \end{aligned}$$

- Interpolation weights can be obtained by means of the EM algorithm

[†]where p_n^* is the unsmoothed probability of an n -gram language model and λ_n is the weight assigned to this model

Kneser-Ney Smoothing

- Kneser-Ney (KN) smoothing is widely considered the most effective smoothing technique for n -gram language models
- KN smoothing is a discounting technique that pays special attention to the number of different words that precede a given one
- Example: consider the bigram “San Francisco”
 - If the training text contains the bigram many times, then the unigram “Francisco” is going to have a high frequency too
 - A standard unigram model would make skewed predictions
 - KN smoothing will consider the frequency of the unigram in relation to possible words preceding it (“Francisco” will have a low probability)

Kneser-Ney Smoothing

Equation for bigram probabilities:

$$p_{\text{KN}}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - \delta, 0)}{\sum_{w'} c(w_{i-1}, w')} + \lambda_{w_{i-1}} p_{\text{KN}}(w_i)$$

where:

- δ is a discount value between 0 and 1
- $\lambda_{w_{i-1}}$ is a normalizing constant
- $p_{\text{KN}}(w_i)$ is the unigram probability:

$$p_{\text{KN}}(w_i) = \frac{N_{1+}(\bullet w_i)}{N_{1+}(\bullet \bullet)}$$

- $N_{1+}(\bullet w_i) = |\{w_{i-1} : C(w_{i-1}, w_i) > 0\}|$ is the number of distinct words w_{i-1} that appear at least once immediately before w_i
- $N_{1+}(\bullet \bullet)$ is the total number of distinct bigram types

Text Generation as Sequential Sampling

Text Generation as Sequential Sampling

A language model defines a probability distribution over sequences of words. Text generation exploits this: at each step we **sample the next word** from the distribution conditioned on the words already produced, then append it and repeat

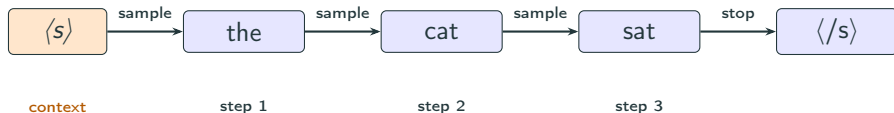
For an n -gram model, the conditioning context is the **last $n - 1$ words**:

$$p(x) \approx \prod_{i=1}^N p(w_i \mid w_{i-n+1}, \dots, w_{i-1})$$

Generation Procedure: Algorithm

1. **Initialize** the context with the sentence-start marker(s) $\langle s \rangle$ (one per conditioning position)
2. **Look up** the probability distribution over all vocabulary words w given the current context
3. **Sample** a word w_t from that distribution
4. **Append** w_t to the output sequence and **update** the context (drop the oldest word if necessary to keep the window size $n - 1$)
5. **Repeat** steps 2–4 until the end-of-sentence marker $\langle /s \rangle$ is sampled, or a maximum length is reached

The Generation Procedure: Bigram example



- Step 1: sample from $p(w \mid \langle s \rangle)$
- Step 2: sample from $p(w \mid \text{the})$
- Step 3: sample from $p(w \mid \text{cat})$
- $\langle /s \rangle$ is sampled $\Rightarrow$ stop

Sampling Strategies

The choice of **sampling strategy** strongly affects the diversity and coherence of the generated text.

Greedy / argmax

Always pick the most probable word:

$$w_t = \arg \max_w p(w \mid \text{ctx})$$

- + Deterministic, locally optimal
- Repetitive, degenerate output

Multinomial sampling

Draw w_t according to the full distribution:

$$w_t \sim p(w \mid \text{ctx})$$

- + Varied, creative output
- Can produce low-probability words

Temperature scaling

Rescale logits before sampling:

$$p_\tau(w) \propto p(w)^{1/\tau}$$

- $\tau \rightarrow 0$ (cold): greedy behaviour
- $\tau = 1$ (neutral): standard sampling
- $\tau > 1$ (hot): more random
- + Tunable diversity

Practical Aspects when Working with n -gram Language Models

Scalability Issues of n -gram Language Models

- Storing all counts for n -grams might require a lot of memory if n is big
- 140 GB of RAM for language model described in https://kheafield.com/papers/edinburgh/estimate_paper.pdf
- Neural Networks offer an alternative where RAM requirements scale with the number of words

Numerical Stability with n -gram Models

- When computing the probability for a long sentence we end up multiplying a lot of probabilities over the n -grams. Each product of two small numbers becomes an even smaller number

$$p(< s > \text{ this is the house } < /s >) = p(\text{this}|< s >)p(\text{is}|\text{this})p(\text{the}|\text{is})p(\text{house}|\text{the})p(< /s >|\text{house})$$

- This may result in numerical underflow

$$0.0001 \cdot 0.0003 \cdot 0.0003 \cdot 0.0003 \cdot 0.0005 = 1.349e - 18$$

Computing in Log Domain

- Computing in log domain allows us to deal with products of probabilities in a smart approach that can avoid numerical problems
- If a number a is positive then $a = \exp(\log(a)) = \log(\exp(a))$
- Therefore we can compute:

$$p_1 \cdot p_2 \cdot p_3 = \exp(\log(p_1 \cdot p_2 \cdot p_3)) = \exp(\log(p_1) + \log(p_2) + \log(p_3))$$

Computing in Log Domain

- If the previous equality is an equality why do we care?
 - Because summing $\log(p_i)$ will not result to numerical underflow but multiplying the p_i might.
- In practice, we will only use probabilities if we need to report them at the end
- The rest of the time, all calculations and storage are made in log space

Evaluating Language Models

Evaluating Language Models

- One of the most common metrics to evaluate a language model is perplexity
- The perplexity of a language model on a test set is defined as the inverse probability of the test set, normalized by the number of words
- The higher the probability the lower the perplexity

Evaluating Language Models

- For a test set $X = w_1, \dots, w_N$ the perplexity of a model h on X is:

$$\text{PP}(h; X) = P_h(w_1, \dots, w_N)^{-\frac{1}{N}}$$

- If we use the chain rule to compute $P_h(w_1, \dots, w_N)$, we have

$$\text{PP}(h; X) = P_h(w_1, \dots, w_N)^{-\frac{1}{N}} = \left(\prod_{i=1}^N P_h(w_i | w_{i-1}, \dots, w_1) \right)^{-\frac{1}{N}}$$

- Taking logarithms:

$$\text{PP}(h; X) = -\frac{1}{N} \sum_{i=1}^N \log P_h(w_i | w_{i-1}, \dots, w_1)$$

Evaluating Language Models

- Note that in the definition of perplexity we used $X = w_1, \dots, w_N$
- Here we mean the concatenation of all the lines in the test set. Since the test set will contain many sentence boundaries we need to include them in the probability computation
- Given the test set
 < s > I am Sam < / s >
 < s > Sam I am < / s >
 < s > I do not like green eggs and ham < / s >
- We represent the whole test set in a single line:
 < s > I am Sam < / s > < s > Sam I am < / s > < s > I do not like green eggs and ham < / s >

The Gutenberg and Brown Corpora

Project Gutenberg is a volunteer-run digital library founded by Michael Hart in 1971, the world's oldest digital library. It provides free e-texts of works whose US copyright has expired, covering literature, history, science, and more:

- Over 70 000 free e-books available at www.gutenberg.org
- Plain-text format makes the collection ideal for NLP research
- Widely used benchmark in computational linguistics since the 1990s

Project Gutenberg: The NLTK Gutenberg Corpus

A curated subset of 18 classic English literary texts, bundled with the Natural Language Toolkit (NLTK) for teaching and quick experimentation

- Included since the early versions of NLTK
- Texts span the 17th–20th century
- Genres: novels, poetry, the Bible, ...
- Total size: $\approx$ 2.6 million words

Quick access:

```
import nltk
nltk.download('gutenberg')

from nltk.corpus import gutenberg
```

Compiled at Brown University by W. Nelson Francis and Henry Kučera, released in 1961 and published in 1967. It was the first large, systematically sampled, machine-readable corpus of English:

- 1 million words of American English
- Sampled from 500 text passages of $\approx 2\,000$ words each
- All texts published in the United States in 1961
- Designed to be balanced: equal representation across genres
- Distributed by the Linguistic Data Consortium (LDC)

- **Word tokenisation:** pre-tokenised, punctuation separated
- **POS tags (original):** 87-tag Brown tagset (e.g. NN, VBD, JJ, ...)
- **POS tags (simplified):** universal 12-tag tagset available via NLTK
- **Sentence boundaries:** explicit
- **Paragraph boundaries:** available in the raw corpus

Brown Corpus: Basic API

Loading & navigation

```
import nltk
nltk.download('brown')          # one-time
from nltk.corpus import brown

# List all 500 file IDs
brown.fileids()                 # ['ca01', ...]

# List the 15 categories
brown.categories()

# Words in a category
brown.words(categories='news')

# Words in a specific file
brown.words(fileids='ca01')

# Sentences
brown.sents(categories='fiction')
```

POS-tagged access

```
# Tagged words: list of (word, tag) pairs
brown.tagged_words(categories='news')
# [('The', 'AT'), ('Fulton', 'NP-TL'), ...]

# Tagged sentences
brown.tagged_sents(categories='news')

# Use the Universal tagset (12 tags)
brown.tagged_words(
    categories='news',
    tagset='universal'
)
# [('The', 'DET'), ('Fulton', 'NOUN'), ...]
```

NOTE: File IDs follow the pattern ca01–cr09, where the letter pair encodes the category

Summary

Summary

- n -gram LMs assign probabilities to word sequences via the Markov assumption
- Parameters are estimated by relative frequency counting
- Zipf's law creates many unseen n -grams; smoothing (Laplace, Lidstone, backoff, Kneser-Ney) gives them non-zero probability
- Perplexity measures model quality on a test set
- Text generation samples words one at a time from the conditional distribution until $\langle /s \rangle$ is produced
- Computations use log space to avoid numerical underflow